

Sistema de Gestão de Restaurantes

Relatório técnico — API REST de gerenciamento de usuários

AUTORES

Rafael Brandão

Felipe Dias Mac Dowell

Rodrigo do Carmo Cruz

Francisco Ferreira

Ana Melissa Lima do Nascimento

CURSO

Pós-Tech em Arquitetura e Desenvolvimento Java

DATA DE ENTREGA

01/09/2026

REPOSITÓRIO

<https://github.com/felipe11dias/postech-restaurantes-app> (público)

CÓDIGO DA ENTREGA

fase-01/camadas/restaurantes/

Sumário

- 1. Introdução
- 2. Definição do problema
- 3. Levantamento de requisitos
 - 3.1 Event Storming
 - 3.2 Mapeamento de demandas e necessidades
 - 3.3 Identificação dos agregados (domínios)
- 4. Arquitetura do sistema
 - 4.1 Abordagem Domain-Driven Design (DDD)
 - 4.2 Definição dos domínios
 - 4.3 Modelagem conceitual
 - 4.4 Resumo da arquitetura
- 5. Documentação do projeto
- 6. Implementação
 - 6.1 Tecnologias e ferramentas
 - 6.2 Fluxos principais
 - 6.3 Tipos de usuários e rotas disponíveis
 - 6.4 Tratamento de erros
 - 6.5 Banco de dados e migrations
 - 6.6 Documentação da API
 - 6.7 Monitoramento
 - 6.8 Testes e qualidade
- 7. Considerações finais
- 8. Artefatos
- 9. Referências

1. Introdução

Este documento descreve as atividades desenvolvidas na Fase 1 do Tech Challenge da Pós-Tech em Arquitetura e Desenvolvimento Java. O objetivo da fase é resolver a necessidade de gerenciamento de usuários de um sistema de restaurantes por meio de uma API REST que centraliza o cadastro, a autenticação e a administração de clientes e proprietários de restaurantes.

A solução foi construída com **Java 21**, **Spring Boot 3.5**, **PostgreSQL 16** e **Docker Compose**, aplicando boas práticas de arquitetura, segurança e organização de código, com foco em uma aplicação escalável, segura e de fácil manutenção.

A entrega oficial usa **arquitetura em camadas**. Em paralelo, o mesmo escopo foi implementado em **arquitetura hexagonal**, sobre a mesma stack, para efeito de estudo e comparação entre as duas abordagens (seção 8).

2. Definição do problema

Restaurantes que utilizam soluções digitais precisam de um mecanismo confiável para gerenciar o cadastro e a autenticação de usuários, garantindo que clientes e proprietários tenham acesso seguro às funcionalidades disponíveis. A ausência de um processo estruturado abre espaço para falhas de segurança, dificulta a administração dos usuários e reduz a confiabilidade do acesso ao sistema.

Diante desse cenário, o projeto entrega uma API REST responsável pelo gerenciamento de usuários — contemplando cadastro, autenticação e controle de acesso por perfil — que serve de base segura, organizada e escalável para as próximas funcionalidades do sistema de gestão de restaurantes.

3. Levantamento de requisitos

3.1 Event Storming

O levantamento de requisitos partiu da identificação dos principais eventos de negócio relacionados ao gerenciamento de usuários. A partir dos fluxos da aplicação, foram mapeados os comandos executados pelos usuários, os eventos gerados e os agregados responsáveis por manter a consistência das informações.

O exercício permitiu compreender o ciclo de vida do usuário no sistema — do cadastro e da autenticação à atualização de dados, alteração de senha e exclusão da conta —, além das regras de autorização por perfil e por posse do recurso.

Comandos

- Cadastrar usuário;
- Realizar login;
- Buscar usuário;
- Atualizar dados do usuário;
- Alterar senha;
- Excluir usuário.

Eventos

- Usuário cadastrado;
- Usuário autenticado;
- Usuário atualizado;
- Senha alterada;
- Usuário removido;
- Acesso autorizado ou negado conforme perfil e permissões.

Agregados

- Usuário;
- Endereço;
- Perfil de acesso (`Role`).

Fluxo principal

- O usuário realiza o cadastro no sistema;
- Efetua a autenticação para obtenção do token JWT;
- Utiliza o token para acessar as funcionalidades protegidas;
- Gerencia os próprios dados, incluindo atualização de informações e alteração de senha;
- Administradores têm permissão para acessar e gerenciar qualquer usuário do sistema.

3.2 Mapeamento de demandas e necessidades

O sistema atende à necessidade de um gerenciamento seguro e eficiente de usuários, permitindo cadastro, autenticação e administração das informações de forma centralizada. Há três perfis de acesso, definidos pela entidade `Role` e persistidos no banco de dados:

Cliente (`ROLE_CUSTOMER`)

Usuário que consome o sistema como cliente. Pode realizar o próprio cadastro, autenticar-se, consultar e atualizar seus dados, alterar a senha e excluir a própria conta.

Proprietário de restaurante (`ROLE_OWNER`)

Representa o responsável por um restaurante. O perfil está previsto na modelagem de autorização da aplicação, mas, na Fase 1, o escopo implementado concentra-se em gerenciamento de usuários, autenticação e autorização — ainda não contempla funcionalidades específicas de gestão de restaurantes.

Administrador (`ROLE_ADMIN`)

Usuário responsável pela administração do sistema. Possui permissões superiores às dos demais perfis e pode consultar, atualizar e excluir usuários diferentes do usuário autenticado, conforme as regras de autorização implementadas.

O cadastro público não permite que um usuário atribua a si próprio o papel `ROLE_ADMIN`. Esse papel é disponibilizado apenas pelo seed administrativo ou por um fluxo autenticado e autorizado.

3.3 Identificação dos agregados (domínios)

O modelo de domínio tem como principal agregado a entidade `User`, que representa a identidade do usuário e concentra os relacionamentos usados pelas regras de negócio.

Agregado `User`

É a raiz do agregado. Reúne os dados de identificação, as credenciais, os papéis de acesso e os endereços do usuário.

Entidade `Address`

Representa os endereços associados a um usuário. Há uma relação de um-para-muitos entre usuário e endereço, sendo o endereço dependente do usuário. Como não há ORM, o `Address` guarda apenas o `id` do usuário (`userId`), e não a entidade inteira — o mesmo vale para o `PasswordResetToken`.

Entidade `Role`

Representa os papéis de acesso usados para autorização. Os papéis são compartilhados entre usuários e relacionados pela tabela associativa `user_roles`, caracterizando uma relação muitos-para-muitos.

Entidade `PasswordResetToken`

Representa os tokens usados na recuperação de senha. Cada token pertence a um usuário, possui prazo de expiração e pode ser utilizado uma única vez.

Relações principais

- `User` → `Address`: 1:N;
- `User` → `Role`: N:N;
- `User` → `PasswordResetToken`: 1:N.

O `User` permanece como núcleo do modelo, enquanto `Address`, `Role` e `PasswordResetToken` apoiam funcionalidades específicas ligadas ao ciclo de vida e à segurança do usuário.

4. Arquitetura do sistema

4.1 Abordagem Domain-Driven Design (DDD)

O projeto utiliza conceitos de Domain-Driven Design para organizar as entidades, as responsabilidades e as regras do domínio de gerenciamento de usuários. O agregado `User` é o núcleo do modelo e se relaciona com `Address` (endereços), `Role` (perfis de acesso) e `PasswordResetToken` (recuperação de senha).

A implementação adota uma **arquitetura em camadas**, com responsabilidades separadas em quatro papéis:

- **Controllers** — expõem a API REST e traduzem requisições e respostas HTTP;
- **Services** — concentram as regras de negócio e a orquestração das operações;
- **Repositories** — acesso ao PostgreSQL com Spring JDBC (`JdbcTemplate`);
- **Entities** — representam o modelo de domínio.

Componentes transversais — segurança com JWT, tratamento de exceções, mapeamento entre objetos (`MapStruct`) e configuração — complementam a estrutura da aplicação.

Persistência sem ORM

O acesso a dados usa **JDBC puro** (`JdbcTemplate`), sem JPA/Hibernate. Foi uma decisão deliberada: o que um ORM faria de forma implícita — cascata de persistência, remoção de órfãos, manutenção da tabela associativa `user_roles` e preenchimento das colunas de auditoria — está escrito como SQL nos repositórios e coordenado na camada de serviço. O agregado `User` continua sendo o único ponto de entrada: papéis e endereços só são gravados através dele.

Regras de arquitetura verificadas no build

As regras de camada não ficam apenas descritas. **Cinco testes de ArchUnit** rodam junto com a suíte e falham o build se forem violadas — por exemplo, se um `controller` passar a acessar um `repository` diretamente, se uma entidade depender de um VO ou de um controller, ou se um repositório deixar de ser uma interface.

4.2 Definição dos domínios

O domínio principal é o **Gerenciamento de Usuários**, responsável pelo cadastro, pela autenticação, pela consulta, pela atualização, pela alteração de senha e pela exclusão de usuários, além das regras de

autorização e do relacionamento dos usuários com seus papéis de acesso e endereços. A partir da implementação, identificam-se os seguintes subdomínios e respectivos limites contextuais.

4.2.1 Domínio de usuários

É o domínio central da aplicação e representa a identidade dos usuários do sistema. Responde pelo cadastro, pela consulta, pela atualização, pela alteração de senha e pela exclusão de usuários, e concentra regras de negócio como:

- validação e unicidade de e-mail e login;
- normalização dos dados de entrada;
- armazenamento seguro da senha utilizando BCrypt;
- associação dos usuários aos seus papéis de acesso;
- associação dos usuários aos seus endereços;
- controle de acesso aos recursos;
- registro das informações de auditoria.

O agregado `User` é a raiz do domínio e mantém as relações com `Role` e `Address`.

4.2.2 Domínio de autenticação e segurança

Responsável pela autenticação dos usuários e pelo controle de acesso às funcionalidades protegidas da API, utilizando Spring Security e JWT. O usuário faz login com suas credenciais e, após a validação (comparação BCrypt pelo `AuthenticationManager`), recebe um token JWT assinado em HMAC-SHA256, com validade de 1 hora, que deve acompanhar as requisições protegidas. A sessão é `STATELESS`: nenhum estado de autenticação é guardado no servidor. Fazem parte deste contexto as funcionalidades de recuperação de senha, com tokens de uso único e validade limitada. A autorização considera os papéis associados ao usuário e também a posse do recurso: operações sobre um usuário específico podem ser executadas pelo próprio usuário ou por um administrador.

4.2.3 Domínio de perfis de acesso

Responsável pela definição dos papéis usados na autorização. Os papéis existentes são:

- `ROLE_CUSTOMER`: representa o cliente;
- `ROLE_OWNER`: representa o proprietário/dono de restaurante;
- `ROLE_ADMIN`: representa o administrador do sistema.

Os papéis são persistidos na tabela `roles` e relacionados aos usuários pela tabela associativa `user_roles`, permitindo que um usuário possua mais de um papel. O `ROLE_ADMIN` tem privilégios administrativos e pode acessar recursos de outros usuários; os usuários comuns acessam os próprios recursos, conforme as regras de autorização.

4.2.4 Domínio de endereços

Responsável pelos endereços associados aos usuários. Um usuário pode ter um ou mais endereços, em uma relação de um-para-muitos entre `User` e `Address`. O endereço depende do usuário ao qual está vinculado e é removido quando o usuário é excluído. O domínio contempla rua, número, complemento, bairro, cidade, estado e CEP.

4.2.5 Domínio de recuperação de senha

Responsável pelo processo de recuperação de senha quando o usuário não possui ou não lembra a senha atual. O fluxo tem duas operações: solicitação de recuperação por e-mail (envio via Spring Mail) e redefinição da senha com um token temporário. O token trafega por e-mail, mas é persistido apenas como **hash SHA-256** — o valor em claro nunca é gravado —, expira em 30 minutos e só pode ser usado uma vez.

Limites contextuais

O Gerenciamento de Usuários é o núcleo do sistema; os demais domínios atuam como contextos de apoio às funcionalidades relacionadas ao usuário:

- **Usuário** → **Perfis de acesso**: define os papéis e as permissões associados ao usuário;
- **Usuário** → **Endereços**: define os endereços pertencentes ao usuário;
- **Usuário** → **Autenticação e segurança**: autentica o usuário e controla o acesso aos recursos protegidos;
- **Usuário** → **Recuperação de senha**: permite recuperar e redefinir a senha por meio de tokens temporários.

Interação entre os domínios

No **cadastro**, o domínio de Usuários recebe os dados do novo usuário, valida as informações, associa os papéis permitidos e registra os endereços informados. No **login**, o domínio de Autenticação valida as credenciais e gera o token JWT, que passa a identificar o usuário autenticado e a aplicar as regras de autorização nas requisições seguintes. Nas operações de **consulta, atualização, alteração de senha e exclusão**, o sistema verifica se o usuário autenticado tem permissão sobre o recurso: usuários comuns operam os próprios dados e o `ROLE_ADMIN` administra qualquer usuário. O domínio de **Recuperação de senha** interage com o de Usuários para localizar o usuário e permitir a troca segura da senha por meio de um token temporário.

4.3 Modelagem conceitual

Na visão conceitual, o sistema é composto por três domínios principais:

- **Usuários** — cadastro, autenticação, atualização, consulta e exclusão de usuários;
- **Perfis de acesso** — gestão dos papéis (*roles*) que controlam as permissões de acesso à aplicação;
- **Endereços** — armazenamento dos endereços associados aos usuários.

O domínio de **Usuários** é o núcleo e interage diretamente com os domínios de Perfis de acesso e de Endereços: durante o cadastro e a manutenção dos usuários, são atribuídos perfis de acesso e vinculados um ou mais endereços. Os domínios de apoio mantêm baixo acoplamento com o núcleo, o que facilita a manutenção do sistema.

4.4 Resumo da arquitetura

A aplicação segue o padrão de **arquitetura em camadas**, com as responsabilidades divididas entre *controller*, *service*, *repository* e *entity*, apoiada por componentes transversais de segurança, tratamento de erros, configuração e mapeamento. O fluxo principal de uma requisição segue a sequência:

Cliente HTTP → Controller → Service → Repository (JDBC) → PostgreSQL

A autenticação JWT e o mecanismo de autorização atuam de forma transversal, protegendo os endpoints que exigem autenticação. As respostas seguem o modelo REST com HATEOAS (links de navegação e paginação). Essa organização mantém as regras de negócio concentradas na camada de serviço, reduz o acoplamento entre as camadas e facilita a manutenção e a evolução da aplicação.

5. Documentação do projeto

A documentação do projeto está organizada neste relatório e complementada no repositório GitHub:

- **Event Storming** — descrito na seção 3.1, com os comandos, os eventos, os agregados e o fluxo principal do gerenciamento de usuários;
- **Arquitetura proposta** — detalhada na seção 4: abordagem DDD, definição dos domínios, modelagem conceitual e fluxo de requisição em camadas;
- **Modelos conceituais** — as entidades `User`, `Address`, `Role` e `PasswordResetToken` e suas relações estão resumidas nas seções 3.3 e 4.2; o esquema físico e as migrations, na seção 6.5;
- **Implementação** — tecnologias, fluxos, rotas, tratamento de erros e testes na seção 6;
- **Documentação da API** — especificação interativa via Swagger/OpenAPI, descrita na seção 6.6;
- **Artefatos complementares** — código-fonte, coleção Postman, scripts de banco e arquivos Docker, listados na seção 8.

O conjunto foi organizado para permitir a compreensão completa do projeto — do levantamento de requisitos às decisões técnicas de implementação.

6. Implementação

O sistema foi desenvolvido com Java 21 e Spring Boot, seguindo a arquitetura em camadas descrita na seção 4 e adotando práticas inspiradas em DDD: entidades de domínio organizadas, responsabilidades separadas entre as camadas e regras de negócio concentradas na camada de serviço. Os requisitos levantados foram implementados nas funcionalidades de cadastro, autenticação, gerenciamento de perfis de acesso, consulta, atualização, alteração de senha e exclusão de usuários, com validações de negócio e controle de acesso conforme o perfil.

6.1 Tecnologias e ferramentas

Recurso	Finalidade
Java 21 · Spring Boot 3.5.11	Linguagem e framework base da aplicação
PostgreSQL 16	Banco de dados relacional
Spring JDBC (<code>JdbcTemplate</code>)	Acesso a dados — SQL escrito à mão, sem ORM (JPA/Hibernate)
Flyway	Versionamento e migração do schema
Spring Security · JWT (<code>jjwt</code> 0.12.6)	Autenticação e autorização; token assinado em HMAC-SHA256
BCrypt	Hash das senhas dos usuários
Spring HATEOAS	Links de navegação e paginação nas respostas REST

Recurso	Finalidade
MapStruct 1.6.3	Mapeamento VO ↔ entidade gerado em tempo de compilação
springdoc-openapi 2.8.17	Documentação Swagger/OpenAPI
Spring Boot Actuator	Endpoints de monitoramento
Spring Mail	Envio do e-mail de recuperação de senha
JUnit 5 · Mockito · AssertJ	Testes de unidade
ArchUnit 1.4.2	Testes automatizados das regras de arquitetura
JaCoCo 0.8.13	Cobertura de testes, com mínimo travado no build
Docker · Docker Compose	Execução da aplicação e do banco

6.2 Fluxos principais

Cadastro e autenticação

- O usuário realiza o cadastro informando dados pessoais, endereço e perfil permitido. O sistema valida os campos (e-mail e CEP têm formato próprio), garante e-mail e login únicos e bloqueia o autocadastro como `ROLE_ADMIN`. Em caso de sucesso, responde **201 Created**, com o cabeçalho `Location` e o corpo trazendo os links do recurso;
- A seguir, o usuário faz login com login e senha e recebe **200** com o token JWT e o prazo de expiração;
- O token passa a acompanhar as requisições protegidas (cabeçalho `Authorization: Bearer ...`).

Gerenciamento de usuários

- O sistema valida as permissões antes de cada operação;
- O usuário consulta e atualiza os próprios dados (`PUT`), troca a senha (`PATCH`, exige a senha atual) ou exclui a conta (`DELETE`) — as três respondem **204 No Content**;
- Administradores executam as mesmas operações sobre qualquer usuário.

Recuperação de senha

- A solicitação (`forgot-password`) responde sempre **202 Accepted**, exista ou não o e-mail informado — não revelar quais contas estão cadastradas evita a enumeração de usuários;
- Se o e-mail existir, um token de uso único é enviado; ele é guardado apenas como hash SHA-256 e expira em 30 minutos;
- Na redefinição (`reset-password`), token inexistente, expirado ou já usado resulta em **400**; em caso de sucesso, a resposta é **204**.

6.3 Tipos de usuários e rotas disponíveis

A API tem controle de acesso baseado em papéis (`Role`) — os tipos principais são `ROLE_CUSTOMER`, `ROLE_OWNER` e `ROLE_ADMIN` — e as rotas são versionadas pelo prefixo `/api/v1`.

Autenticação

Método	Rota	Descrição	Acesso
POST	<code>/api/v1/auth/login</code>	Realiza a autenticação e retorna o token JWT	Público

Método	Rota	Descrição	Acesso
POST	/api/v1/auth/forgot-password	Solicita a recuperação de senha	Público
POST	/api/v1/auth/reset-password	Redefine a senha utilizando o token de recuperação	Público

Usuários

Método	Rota	Descrição	Acesso
POST	/api/v1/users	Realiza o cadastro de um novo usuário	Público
GET	/api/v1/users	Lista usuários de forma paginada	Autenticado
GET	/api/v1/users?name={nome}	Busca usuários por nome	Autenticado
GET	/api/v1/users/{id}	Consulta um usuário pelo UUID	Próprio usuário ou ROLE_ADMIN
PUT	/api/v1/users/{id}	Atualiza os dados do usuário	Próprio usuário ou ROLE_ADMIN
PATCH	/api/v1/users/{id}/password	Altera a senha do usuário	Próprio usuário ou ROLE_ADMIN
DELETE	/api/v1/users/{id}	Exclui um usuário	Próprio usuário ou ROLE_ADMIN

O cadastro é público, mas não permite que o usuário atribua a si próprio o papel `ROLE_ADMIN`. As rotas por `id` usam `@PreAuthorize` combinando `hasRole('ADMIN')` com uma verificação de posse (`@userSecurity.isSelf`): o usuário comum só alcança o próprio registro; o administrador alcança qualquer um. As rotas de coleção (`GET /users` e a busca por nome) exigem apenas autenticação. A busca por nome é parcial e não diferencia maiúsculas de minúsculas.

A listagem aceita os parâmetros de paginação `page`, `size` (padrão 20) e `sort` (padrão `name`), e devolve a resposta paginada no formato HATEOAS.

6.4 Tratamento de erros

Toda resposta de erro segue o padrão **ProblemDetail (RFC 7807)**, nativo do Spring 6, com um `type` estável por categoria — para o cliente identificar o erro de forma programática, e não pelo texto do título — e um `timestamp`. Erros de validação agregam os campos inválidos em `errors`. Mensagens internas (stack traces, texto do parser de JSON) não vão para o cliente: ficam no log, e a resposta traz uma descrição genérica.

Status	Quando ocorre
400	Campo inválido, corpo ausente ou malformado, <code>id</code> fora do formato UUID, senha inválida, token de recuperação inválido
401	Credenciais incorretas no login ou token ausente/ inválido
403	Acesso a recurso de outro usuário; autocadastro com papel <code>ROLE_ADMIN</code>
404	Recurso não encontrado

Status	Quando ocorre
409	E-mail ou login já cadastrado
500	Erro não previsto — registrado em log, com resposta genérica

6.5 Banco de dados e migrations

O schema é versionado com Flyway e aplicado automaticamente na subida da aplicação. São cinco tabelas: `users`, `addresses`, `roles`, a associativa `user_roles` e `password_reset_tokens`. `email` e `login` são `UNIQUE`; a exclusão do usuário remove em cascata endereços, vínculos de papel e tokens (`ON DELETE CASCADE`). Todas as tabelas têm as quatro colunas de auditoria (`created_at`, `last_updated_at`, `created_by`, `last_updated_by`), preenchidas pela aplicação com o login de quem fez a gravação (ou `system` nos seeds e no autocadastro).

Migration	O que faz
V1	Cria <code>users</code> , <code>roles</code> , <code>user_roles</code> e <code>addresses</code> ; faz o seed dos três papéis
V2	Insere usuários de demonstração (um dono, um cliente) com papéis e endereços
V3	Insere o usuário administrador de demonstração
V4	Cria a tabela de tokens de recuperação de senha
V5	Acrescenta as colunas de auditoria em todas as tabelas
V6	Converte as chaves primárias de <code>BIGINT</code> sequencial para <code>UUID</code> aleatório, para impedir a enumeração de recursos pelas rotas

Para subir tudo localmente (banco + aplicação, com as migrations aplicadas):

```
cd fase-01/camadas/restaurantes && docker compose up --build
```

6.6 Documentação da API

A aplicação disponibiliza documentação interativa via Swagger/OpenAPI:

- **Swagger UI:** `/swagger-ui.html`
- **OpenAPI JSON:** `/v3/api-docs`

Pelo Swagger é possível autenticar com o token JWT obtido no endpoint de login e executar os endpoints protegidos diretamente pela interface.

6.7 Monitoramento

O Spring Boot Actuator expõe três endpoints, com níveis de proteção distintos:

- `/actuator/health` e `/actuator/info` — públicos;
- `/actuator/metrics` — exige autenticação.

Os detalhes do `health` só aparecem para requisições autenticadas (`show-details: when-authorized`).

6.8 Testes e qualidade

A versão em camadas tem **252 testes** (36 classes) cobrindo entidades, VOs com Bean Validation, utilitários, mappers, os repositórios JDBC, a camada de segurança, os services, os controllers e o assembler de links. A versão hexagonal tem 290 testes.

A cobertura é de **100% em todos os contadores do JaCoCo** — instruções, desvios, linhas, complexidade, métodos e classes —, sem exclusões. E não é só um número no relatório: `jacoco:check` roda na fase `verify` exigindo `1.00` em cada contador, então o build falha se a cobertura cair. Somam-se a isso as cinco regras de ArchUnit da seção 4. Para conferir:

```
cd fase-01/camadas/restaurantes && mvn clean verify
```

7. Considerações finais

O levantamento de requisitos por meio do Event Storming ajudou a identificar os principais processos do sistema, compreender o fluxo das funcionalidades e organizar os elementos do domínio. Os conceitos inspirados em Domain-Driven Design orientaram a definição das responsabilidades das entidades e a organização da arquitetura.

Os principais desafios do desenvolvimento estiveram na autenticação com JWT, no controle de acesso por perfis e por posse do recurso, na troca do ORM por SQL escrito à mão e em chegar a 100% de cobertura sem recorrer a exclusões. Foram superados com a adoção de boas práticas de desenvolvimento, arquitetura em camadas, versionamento do banco com Flyway e uso dos recursos do ecossistema Spring.

A cobertura de 100% e as regras de ArchUnit ficam verificadas no `mvn verify`, funcionando como portões de qualidade: se a cobertura cair ou uma regra de camada for violada, o build falha. Em paralelo à entrega em camadas, o mesmo escopo foi construído em arquitetura hexagonal, sobre a mesma stack, permitindo comparar as duas abordagens tendo a arquitetura como única variável.

O resultado é uma API REST segura, organizada e escalável para o gerenciamento de usuários de um sistema de restaurantes. A solução amplia o controle sobre o acesso à aplicação, centraliza o gerenciamento dos usuários e estabelece uma base sólida para as próximas etapas do projeto.

8. Artefatos

Os seguintes artefatos compõem a entrega do projeto:

Artefato	Descrição	Localização
Código-fonte	Implementação completa da API REST desenvolvida em Java com Spring Boot.	<code>github.com/felipe11dias/postech-restaurantes-app</code>
Código da aplicação	Diretório com a implementação da aplicação: controllers, services, repositories, entidades e demais componentes.	<code>fase-01/camadas/restaurantes/</code>
Relatório técnico	Este documento — descrição do projeto, arquitetura, modelagem, implementação e decisões técnicas.	<code>fase-01/relatorios/camadas/</code>
Versão hexagonal	Mesmo escopo implementado em arquitetura hexagonal, sobre a mesma stack, para estudo comparativo.	<code>fase-01/hexagonal/restaurantes/</code>
Documentação da API	Documentação interativa dos endpoints com Swagger/OpenAPI.	<code>/swagger-ui.html</code> (aplicação em execução)
Coleção Postman	47 requisições cobrindo sucesso e falha de todas as rotas.	<code>fase-01/camadas/restaurantes/postman/Restaurants.postman_collection.json</code>
Prints da API	47 imagens, uma por requisição da coleção, com o status esperado no nome do arquivo.	<code>fase-01/camadas/restaurantes/postman/prints/</code>
Scripts de banco de dados	Migrations Flyway V1 a V6 (criação, seeds, auditoria e conversão para UUID).	<code>src/main/resources/db/migration/</code>
Docker	Dockerfile multi-stage (build Maven → runtime jre-alpine) e docker-compose.yml (aplicação + PostgreSQL).	Diretório raiz da aplicação
Relatório de cobertura	Relatório HTML do JaCoCo, gerado por <code>mvn verify</code> .	<code>target/site/jacoco/index.html</code>

9. Referências

- EVANS, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley, 2003.
- FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.
- Spring. *Spring Boot Documentation*. Disponível em: <https://docs.spring.io/spring-boot/index.html>.
- Spring. *Spring Security Reference*. Disponível em: <https://docs.spring.io/spring-security/reference/>.
- Spring. *Spring HATEOAS Reference*. Disponível em: <https://docs.spring.io/spring-hateoas/docs/current/reference/html/>.
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>.

- Flyway. *Flyway Documentation*. Disponível em: <https://documentation.red-gate.com/flyway>.
- Docker Inc. *Docker Documentation*. Disponível em: <https://docs.docker.com/>.
- OpenAPI Initiative. *OpenAPI Specification*. Disponível em: <https://swagger.io/specification/>.
- IETF. *RFC 7807 — Problem Details for HTTP APIs*. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7807>.
- JaCoCo. *Java Code Coverage Library*. Disponível em: <https://www.jacoco.org/jacoco/>.
- ArchUnit. *ArchUnit User Guide*. Disponível em: https://www.archunit.org/userguide/html/ooo_Index.html.
- GitHub. *Repositório do projeto — Tech Challenge: Sistema de Gestão de Restaurantes*. Disponível em: <https://github.com/felipe11dias/postech-restaurantes-app>.

Acesso a todos os links em 28 ago. 2026.